

Experiment No. 3

Aim

Socket Programming for Transport Layer Packets (TCP Client)

Theory

In a client-server architecture, the **client** acts as the active process that initiates communication by sending a request to a passive server running on a known IP address and port number.

At the Transport Layer, the TCP Client provides **Active Open** socket functionality. It guarantees data delivery by using sequence numbers, acknowledgments, and checksums over a virtual connection stream.

Client-Side Execution Steps:

1. **Socket Creation (socket()):** The client initializes an endpoint socket specifying the address family (AF_INET for IPv4) and protocol type (SOCK_STREAM for TCP).
2. **Server Addressing:** The client configures a socket address structure (sockaddr_in) specifying the target server's destination **IP Address** and **Port Number**.
3. **Initiating Connection (connect()):** The client calls connect(), which triggers the **TCP Three-Way Handshake** with the listening server:
 - **SYN:** Client sends a synchronization packet to the server.
 - **SYN-ACK:** Server responds with synchronization and acknowledgment.
 - **ACK:** Client sends an acknowledgment back to confirm the connection.
4. **Data Transmission (send() / recv() or Streams):** Once established, the client sends requests or reads responses through full-duplex socket streams.
5. **Termination (close()):** Upon completing the interaction, the client issues a close() call to gracefully close the socket connection and release kernel resources.

Name : Parth Vishnu PRN : 24070521279

```
lab-3pc-27@lab-3pc-27-OptiPlex-3280-AIO: ~  
lab-3pc-27@lab-3pc-27-OptiPlex-3280-AIO:~$ vim client1279.c  
lab-3pc-27@lab-3pc-27-OptiPlex-3280-AIO:~$ gcc client1279.c -o client  
lab-3pc-27@lab-3pc-27-OptiPlex-3280-AIO:~$ ./client  
Enter message: hello  
Echo from server: hello  
Enter message: hiii  
Echo from server: hiii  
Enter message: parth  
Echo from server: parth  
Enter message:
```

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <netinet/in.h>

#define PORT (10000 + 270) // Replace PRN with your PRN value
#define BUF_SIZE 1024

int main()
{
    int sock;
    struct sockaddr_in server;
    char buffer[BUF_SIZE];

    // Create socket
    socket(AF_INET, SOCK_STREAM, 0);

    // Prepare server address
    server.sin_family = AF_INET;
    server.sin_port = htons(PORT);
    server.sin_addr.s_addr = INADDR_ANY;

    // Connect to server
    connect(sock, (struct sockaddr *)&server, sizeof(server));

    while (1)
    {
        printf("Enter message: ");
        fgets(buffer, BUF_SIZE, stdin);

        send(sock, buffer, strlen(buffer), 0);

        memset(buffer, 0, BUF_SIZE);

        read(sock, buffer, BUF_SIZE);

        printf("Echo from server: %s", buffer);
    }

    close(sock);

    return 0;
}
```

Conclusion

The TCP Client socket was successfully constructed at the Transport Layer. Using the active open mechanism (socket, setting server parameters, and connect), the client successfully established a connection with the server via the TCP Three-Way Handshake and enabled reliable stream-based data transmission.

